Aula 07: Benchmarking e Revisão da Unidade I

Avaliação de Desempenho (PC010029)

Prof. Dr. Helvecio Bezerra Leal Neto

Universidade Federal do Oeste do Pará

22 de setembro de 2026

Roteiro da aula

Bloco 1 — Benchmarking

- O que é e por que padronizar
- Por dentro: os algoritmos do LINPACK, STREAM e Dhrystone
- Das suítes SPEC às ferramentas que vocês usam
- Métricas, armadilhas e o experimento bem feito

Bloco 2 — Revisão para a avaliação

- O mapa da disciplina: Aulas 01 a 07
- Métrica, parâmetro e carga
- Teoria de filas: Little, ρ, Kendall, M/M/1
- Simulação e aferição: o que cobrar de cada uma
- Exercício resolvido e guia de estudo

Na aula anterior (Aula 06)

- Aferição mede um sistema **real** enquanto ele processa uma carga.
- A carga real é fiel, mas não se repete; por isso usamos **carga sintética**.
- O **monitor** observa, registra e resume o que aconteceu.
- Observar custa: *overhead* e resolução limitam o que se pode ver.
- Monitor de hardware mede de fora; o de software é flexível, mas pesa no sistema.

A pergunta que ficou: se cada um mede a sua máquina com a sua carga, como comparar duas máquinas?

Vocabulário da aula

Benchmarks e algoritmos

Benchmark	programa-padrão de teste
Benchmarking	comparar via benchmark
Kernel	núcleo de um programa real
Sintético	imitação de uma carga
Suíte	vários programas, uma nota
LINPACK	resolve $Ax = b$ (Gauss)
DAXPY	$y \leftarrow \alpha x + y$, rotina da BLAS
Cache	memória pequena e rápida
STREAM	mede a banda da memória
HPL	o LINPACK do TOP500
SPEC, TPC	consórcios de benchmarks
SPECspeed	tempo; SPECrate: vazão
Ratio game	ranking muda com a base

Medidas e experimento

FLOPS	op. de ponto flutuante/s
M, G, T, P, E	10^6, 10^9, 10^{12}, 10^{15}, 10^{18}
MIPS	milhões de instruções/s
DMIPS	MIPS do Dhrystone
MWIPS	milhões de Whetstones/s
IOPS	operações de E/S por segundo
Latência	tempo de uma operação
KiB, MiB	2^{10} e 2^{20} bytes
Jitter	variação do atraso
TCP, UDP	transporte com/sem garantia
MTU	maior pacote do enlace
Transiente	início, antes do regime
Réplica	repetição do mesmo cenário

Duas máquinas, uma decisão

A situação: a UFOPA vai comprar um servidor. Dois candidatos, mesmo preço. **Qual você compra?**

	Servidor A	Servidor B
Frequência	3,2 GHz	2,4 GHz
Núcleos	8	16
Memória	32 GB	64 GB
Disco	2 TB	2 TB

Catálogo fictício, para o argumento.

A turma se divide: metade aponta A ("tem mais GHz"), metade aponta B ("tem mais núcleos"). **As duas respostas são chutes**, e por um motivo só:

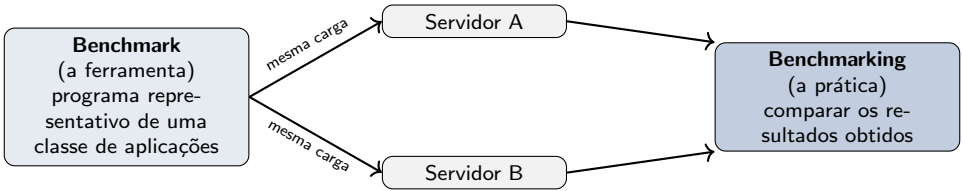
- Se o nosso sistema passa boa parte do tempo esperando o disco, os 3,2 GHz de A não vão ser usados.
- Se a nossa carga é um cálculo que roda em um fluxo só, os 16 núcleos de B ficam parados.

O catálogo descreve **a máquina**. Nós precisamos saber da **máquina fazendo o nosso trabalho**.

Ideia-chave: Sem a mesma carga e o mesmo procedimento, não há comparação.

Benchmarking é uma forma de aferição

Benchmarking mede um sistema **existente**, sob uma carga **definida**, capturando seu desempenho naquele momento — é aferição, com carga e procedimento padronizados.



Condição inegociável, herdada da Aula 06: a execução do benchmark **não pode influenciar** o resultado medido.

Para que se padroniza

O problema da variação

Sem padronização, cada teste usa cargas próprias e inviabiliza a comparação justa entre computadores.

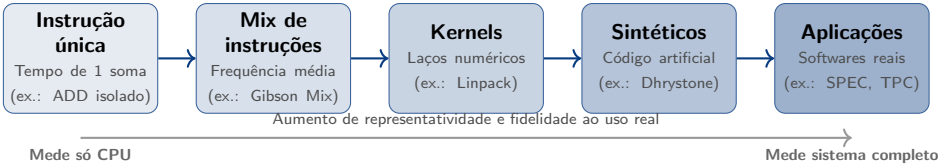
Ponto de referência

Ao fixar **carga** e **método**, o benchmark estabelece uma **referência comum** e reproduzível.

Objetivos práticos

- **Comparar**: confrontar plataformas sob condições rigorosamente idênticas.
- **Otimizar**: isolar o impacto real de melhorias em hardware ou software.
- **Decidir**: orientar a aquisição de equipamentos com métricas confiáveis.

Cinco gerações de carga de teste



- **Instruções e Mix (Gibson)**: medem tempo de operações isoladas ou médias ponderadas.
- **Kernels e Sintéticos (Dhrystone)**: executam laços matemáticos ou código artificial simplificado.
- **Aplicações (SPEC, TPC)**: rodam programas reais para testar processador, memória e I/O.

Fonte: material de Aferição, p. 5-8.

Por que a Carga Sintética Venceu no Benchmarking

Problemas da Carga Real

- **Irrepetível:** impede a comparação justa entre sistemas.
- **Dados sensíveis:** expõe informações privadas de usuários.
- **Rígida:** dificulta a simulação de cenários extremos.

Vantagens da Sintética

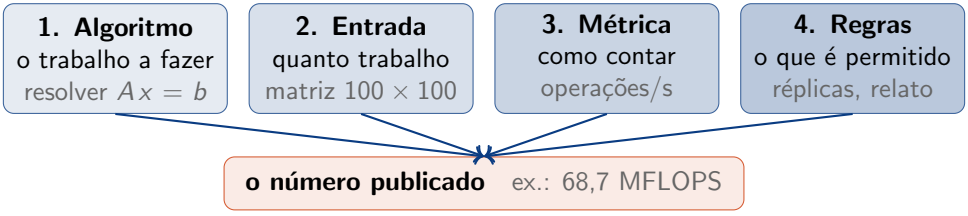
- **Reprodutível:** garante testes idênticos e auditáveis.
- **Portável e segura:** roda em qualquer ambiente sem dados reais.
- **Instrumentada:** embute métricas diretas de vazão e latência.

Princípio do Benchmarking: comparar sistemas exige condições idênticas. A carga sintética garante o controle experimental no sistema real.

Fonte: material de Aferição, p. 5-8.

Por dentro de um benchmark: quatro peças

Todo benchmark, do LINPACK de 1983 ao aplicativo do celular, tem quatro peças:



A pergunta dos próximos slides: que conta este benchmark faz, e que parte da máquina essa conta põe para trabalhar?

Ideia-chave: O algoritmo escolhe qual recurso vai ser medido. Para entender o número, entenda o algoritmo.

LINPACK por dentro: eliminação de Gauss

O LINPACK (Dongarra, 1983) resolve $Ax = b$ como em Álgebra Linear: **escalonar** e **substituir de baixo para cima**.

Em miniatura: 2×2

$$\left[\begin{array}{cc|c} 2 & 1 & 5 \\ 4 & 3 & 11 \end{array}\right] \xrightarrow{L_2 - 2L_1} \left[\begin{array}{cc|c} 2 & 1 & 5 \\ 0 & 1 & 1 \end{array}\right]$$

Daí $y = 1$ e $2x + 1 = 5$, logo $x = 2$.

Em tamanho real: $n = 100$

```
for k in range(n):                  # cada pivô
  for i in range(k+1, n):           # linhas abaixo
    m = A[i][k] / A[k][k]
    A[i][k:] -= m * A[k][k:] # y = y + a*x
```

A última linha é a rotina `DAXPY` da biblioteca **BLAS**: é nela que o LINPACK passa quase todo o tempo. Três laços aninhados: **dobrar n multiplica o trabalho por 8**.

Fonte: Jain, *The Art of Computer Systems Performance Analysis*, p. 47.

LINPACK: do cronômetro ao MFLOPS

O algoritmo é conhecido, então o **número de operações** também é, antes de rodar:

$$\frac{2}{3}\,n^3 + 2\,n^2 \quad \xrightarrow{n=100} \quad \text{cerca de } \mathbf{687} \text{ mil operações}$$

Exemplo resolvido

Se a máquina resolve em $t = 0{,}01$ s:

$$\text{MFLOPS} = \frac{686\,667}{0{,}01 \times 10^6} \approx \mathbf{68{,}7}$$

O que o número não diz

A matriz tem 80 KB e **cabe na cache**: quase não usa RAM, disco nem rede. Não prevê um banco de dados.

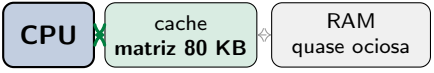
Hoje: o **HPL** aumenta n até encher a memória de um supercomputador. É o número do ranking **TOP500**, cujo topo já passa de 1 EFLOPS (10^{18} operações/s).

Fontes: Jain, p. 47; `https://top500.org`.

Mesma conta, outro recurso: o STREAM

O **STREAM** (McCalpin, 1995) faz quase a conta do DAXPY, `a[i] = b[i] + q*c[i]`, mas com vetores **muito maiores que a cache**.

LINPACK



cada número é **reusado** cerca de 100 vezes:
a CPU não para ⇒ mede **FLOPS**

STREAM



cada número é usado **uma vez só**:
a CPU espera a memória ⇒ mede **GB/s**

Ideia-chave: O recurso medido depende da conta **e** do tamanho dos dados: o gargalo muda de lugar.

Whetstone e Dhrystone: o programa "médio"

Um programa **artificial** que imita a **mistura de operações** de programas reais:

	Whetstone (1972)	**Dhrystone** (1984)
Imita	programas científicos	programas de sistema
O laço faz	seno, exponencial, raiz, vetores	inteiros, *strings*, ponteiros
Ponto flutuante	muito	nenhum: o *dry* do nome
Métrica	MWIPS	DMIPS (Dhrystones/s $\div$ 1757)

Quando o teste vira alvo

Os compiladores aprenderam a reconhecer esses programas e a pular as contas cujo resultado ninguém usava. O número subia; os programas reais, não. (Jain, p. 47-48)

Ideia-chave: Quando uma medida vira meta, ela deixa de ser uma boa medida.

Do programa isolado à suíte: a resposta do SPEC

O consórcio **SPEC** (fabricantes e universidades) atacou os três problemas de uma vez:

Um número só engana
⇒ **suíte** com dezenas de programas

Programa artificial é "otimizável"
⇒ **programas reais**

Cada um mede do seu jeito
⇒ **regras** e relatório revisado

Dentro do SPEC CPU 2017

`gcc`: compila programas C
`xz`: comprime dados
`omnetpp`: **simula eventos discretos** (Aula 04!)
`blender`: renderiza cenas 3D

Duas leituras: **SPECspeed** (uma cópia: **tempo de resposta**) e **SPECrate** (várias cópias juntas: **vazão**).

Fontes: Jain, p. 51-52 e p. 535; `https://www.spec.org`.

SPEC: como a suíte vira um número

Passo 1. Para cada programa: **razão** $= t_{\text{referência}} \,/\, t_{\text{testada}}$ (2 = duas vezes mais rápida).
Passo 2. Resumo: a **média geométrica** das razões.

Programa	Razão
P1	1,0
P2	2,0
P3	**20,0**

Aritmética: $(1+2+20)/3 = \mathbf{7{,}7}$: puxada por P3.
Geométrica: $\sqrt[3]{1\times 2\times 20} = \mathbf{3{,}4}$: resiste ao extremo.

Por que a geométrica? Razões **multiplicam**: 2 vezes mais rápido num programa e 2 vezes mais lento em outro é empate. A geométrica dá $\sqrt{2\times 0{,}5} = 1$; a aritmética, 1,25. Guarde esse 1,25.

Exemplo didático. Fonte: Jain, p. 51-52.

Ferramentas que vocês provavelmente já viram

Leia qualquer ferramenta com duas perguntas: **que conta ela faz?** e **em que unidade responde?**

Ferramenta	**O que ela executa**	**Responde em**
Geekbench (CPU)	comprime, abre HTML, filtra imagens	pontos
Cinebench (CPU)	renderiza uma cena 3D	pontos
3DMark (GPU)	cenas de jogo em tempo real	pontos e quadros/s
CrystalDiskMark (disco)	lê e grava em sequência e ao acaso	MB/s
Speedtest (internet)	troca dados com um servidor próximo	Mbit/s e ping

Ideia-chave: Pontos não são unidade física: só compare na mesma ferramenta e na mesma versão.

No servidor: “o sistema está lento”. Lento onde?

Antes de medir, escolha **qual recurso** é o sistema sob teste:

Recurso	Ferramenta	O que ela executa	Métrica
CPU	`sysbench`	busca números primos por divisão	eventos/s
Disco	`fio`	leituras aleatórias de 4 KiB	IOPS, latência
Rede	`iperf3`	fluxo TCP ou UDP entre máquinas	Mbit/s, perda
Banco	`pgbench`	transações curtas (estilo TPC-B)	transações/s
Web	`wrk`, JMeter	muitos clientes ao mesmo tempo	req./s, latência

Para rodar centenas de testes de uma vez: **Phoronix Test Suite**.

Ideia-chave: Não existe benchmark universal: cada um fixa um objetivo, uma métrica e uma carga.

Representatividade: o critério que decide tudo

Uma carga não representativa torna o estudo **incorreto ou inaceitável** — não importa quão bem conduzido esteja o resto.

Quantitativo

O **volume** bate com o uso real. Abaixo do esperado, **subdimensiona** o sistema; muito acima, **superdimensiona**.

Qualitativo

O **perfil** bate: mesmo tempo decorrido, mesma demanda por recursos, mesma sequência de uso.

Atualidade

O usuário otimiza a sua demanda e novos sistemas geram novas cargas: é preciso monitorar continuamente.

Nível de carga

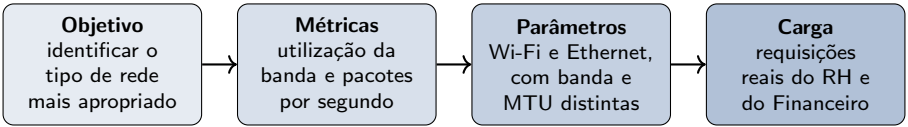
Até a capacidade (melhor caso), além dela (pior caso) ou no nível da operação real (caso típico). Declare qual você escolheu.

Fonte: Livro AD, 1.4.3, p. 25; material de Aferição, p. 32-35.

Ideia-chave: Carga errada, conclusão errada — a mesma regra da Aula 06.

Exemplo resolvido: a rede da folha de pagamento

Problema: qual tipo de rede é mais adequado ao sistema de folha de pagamento da empresa?



Troque a carga e a conclusão muda: alimentar o mesmo estudo com o perfil de uma biblioteca on-line levaria a um dimensionamento completamente diferente — e errado.

Fonte: Livro AD, 7 Quantidades, p. 26.

As métricas que o benchmark reporta

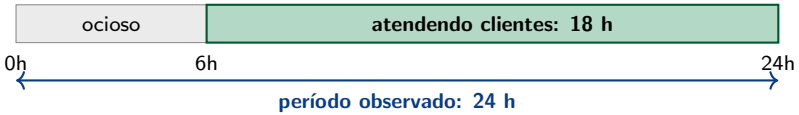
Antes de ler qualquer resultado, saiba **qual grandeza** o benchmark mediu.

Métrica	O que mede	Unidade típica	Domínio
Utilização	Fração do tempo ocupado	% ou fração de 0 a 1	CPU, disco, enlace
Vazão	Pedidos atendidos por unidade de tempo	MIPS, MFLOPS, bit/s, consultas/s	CPU, rede, banco
Tempo de resposta	Da chegada ao fim do atendimento	ms ou s	Sistema interativo
Atraso fim a fim	Pacote ao longo de toda a rota	ms	Rede
Jitter	Variação desse atraso	ms	Voz e vídeo

Referência concreta: atrasos acima de **300 ms** normalmente inviabilizam voz e vídeo; jitter alto também. Fonte: pdfcoffee-ADS, p. 26-27 e p. 123-124.

Exemplo resolvido: utilização do servidor Web

O cenário: o servidor fica **ligado** 24 horas por dia, mas só **atende clientes** das 6h às 24h.



$$U = \frac{\text{tempo ocupado}}{\text{tempo total}} = \frac{18}{24} = \mathbf{75\%}$$

Cuidado com a leitura. U responde "quanto o recurso trabalhou", e **não** "quanto o usuário esperou". São perguntas diferentes: se esta fosse uma M/M/1 com $\rho = 0{,}75$, a Aula 03 já nos daria a outra resposta — em média **3 requisições** no sistema, e cada uma passando **3/4** do seu tempo parada na fila.

Fonte: pdfcoffee-ADS, p. 26-27.

O *ratio game* (1/2): os mesmos tempos, duas manchetes

Dois fabricantes recebem **as mesmas medições**. Cada um escreve o seu relatório.

Tempo medido	**Bench. 1**	**Bench. 2**
Sistema Z	**10 s**	40 s
Sistema Y	20 s	**20 s**

Menor é melhor: **Z vence o 1**, **Y vence o 2**.

Cada relatório usa a **razão** do SPEC: tempo da base ÷ tempo do sistema. Acima de 1, o sistema é melhor que a base.

Relatório do fabricante de Y (base: Z)

Bench. 1: $10 \div 20 = 0{,}5$
Bench. 2: $40 \div 20 = 2$
Média: $(0{,}5 + 2) \div 2 = \mathbf{1{,}25}$

Manchete: "Y é 25% mais rápido"

Relatório do fabricante de Z (base: Y)

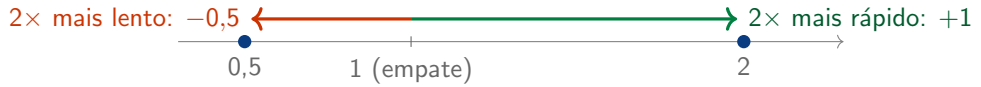
Bench. 1: $20 \div 10 = 2$
Bench. 2: $20 \div 40 = 0{,}5$
Média: $(2 + 0{,}5) \div 2 = \mathbf{1{,}25}$

Manchete: "Z é 25% mais rápido"

Mesmos números, nenhuma conta errada, conclusões opostas. **Onde está o truque?**

O *ratio game* (2/2): o truque e a defesa

O truque está na média aritmética. Ela não trata as razões com justiça: ser 2 vezes mais rápido **soma 1**, mas ser 2 vezes mais lento **tira só 0,5**. O ganho pesa o dobro da perda.



Defesa 1: média geométrica

$\sqrt{0{,}5 \times 2} = 1$, com qualquer base: **empate**. É por isso que o SPEC a usa.

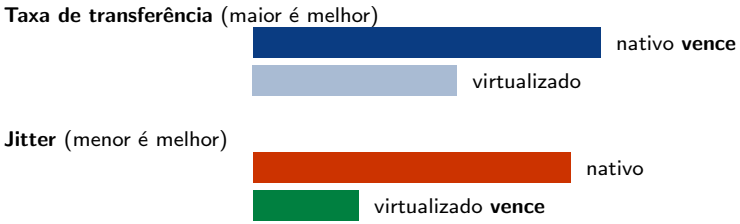
Defesa 2: tempos na *sua* carga

Sua carga roda	**Z**	**Y**	**vence**
1 e 2, uma vez cada	50 s	40 s	Y
3 vezes o 1 para cada 2	70 s	80 s	Z

Ideia-chave: Escolher a métrica e a base já é, em parte, escolher a conclusão.

A métrica decide o veredito

Experimento de virtualização: as mesmas duas máquinas, duas métricas, dois vencedores. **Esquema qualitativo, sem escala:** o comprimento das barras não é dado, só indica quem vence.



Um relatório que citasse só a taxa de transferência concluiria "nativo é sempre melhor". Para uma aplicação de vídeo, isso é falso: ali quem manda é o jitter.

Fonte: pdfcoffee-ADS, p. 172-173.

Por que a mesma máquina dá dois números?

Você mede a mesma máquina duas vezes: **180 ms**, depois **240 ms**. A máquina não mudou; mudou **o que estava em volta dela**.

Fonte	Exemplo	Como controlar
Interferência	antivírus ou backup rodando junto	máquina **dedicada**
Aquecimento	cache vazia, CPU quente que desacelera	descartar o **transiente**
Acaso	escalonador do SO, tráfego da rede	**repetir**: várias réplicas

E anote enquanto mede: máquina e versões, carga, duração e número de réplicas. Sem isso ninguém repete o experimento, nem você daqui a três meses.

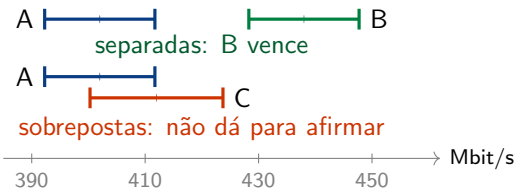
Ideia-chave: Variação não controlada vira uma falsa diferença entre sistemas.

Repetir e comparar: uma rodada é uma amostra

Exemplo: qual de dois roteadores Wi-Fi entrega mais vazão? Só o roteador muda.

```
# no servidor:
iperf3 -s
# no cliente, 5 réplicas de 60 s:
for i in 1 2 3 4 5; do
  iperf3 -c <IP> -t 60 >> A.txt
done
```

Média de cada roteador, com **faixa de 95%**:



Valores fictícios. O livro relata assim: "com 95% de certeza, a média do tempo de resposta está entre 185,46 e 194,54 ms". Fonte: Livro AD, cap. 9 e 10.

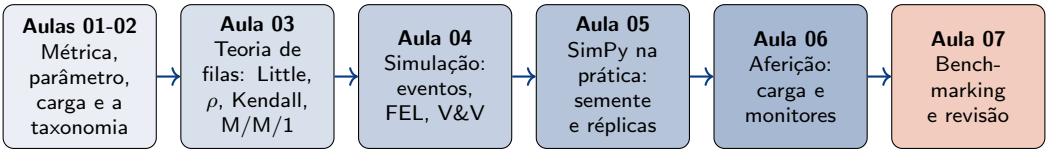
Ideia-chave: Como na Aula 05: uma rodada é uma amostra, não uma medida.

Intervalo de 10 minutos

10 min

Na volta: **revisão completa da disciplina**,
da Aula 01 até aqui — o que cai na avaliação.

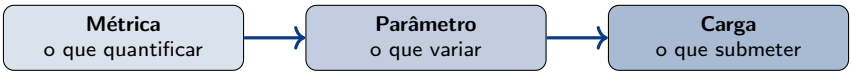
Onde estivemos: o mapa da disciplina



Um só fio: **escolher a métrica** e **aplicar a técnica adequada** — as métricas são as mesmas nas três técnicas

Revisão: o tripé de todo estudo

[Aulas 01 e 02]



- **Métrica:** critério que quantifica o desempenho. Ex.: tempo para responder a um pedido de compra.
- **Parâmetro:** o que afeta a métrica. Ex.: nº de itens do pedido, hardware do servidor. Só entra se afetar a métrica.
- **Duas famílias: de sistema** (CPU, memória, banda) e **de carga** (nº de usuários, tamanho dos pedidos). O que se decide **variar** é um **fator**; os valores que ele assume são os **níveis** (10, 100, 1000 usuários).
- **Carga:** a lista de requisições feitas ao sistema — quantas e de que tipo.

Passos do estudo (Jain): objetivos e limites → serviços e resultados → métricas → parâmetros → carga e técnica → executar, analisar e apresentar.

Revisão: as quatro métricas fundamentais

[Aulas 02 e 03]

Tempo de resposta $E[s]$

Da chegada até o fim do atendimento:
$E[s] = E[w] + E[x]$ (espera mais serviço).

Utilização U

Fração do tempo em que o recurso fica ocupado.
Numa M/M/1, $U = \rho$.

Vazão X

Taxa de saídas: um número só, do sistema inteiro, e sempre média de longo prazo. Em regime estável, dois caminhos dão o mesmo número: $X = \lambda = m\,\rho\,\mu$.

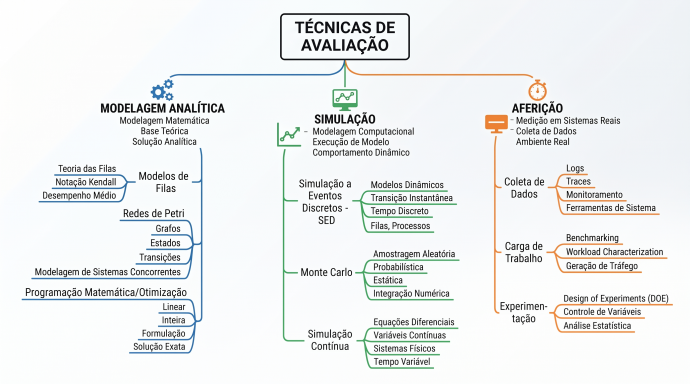
Tamanho da fila $E[n_w]$

Quantos aguardam atendimento, sem contar quem já está em serviço.

O gargalo é o recurso de maior utilização: é ele que limita a vazão do sistema inteiro.

Revisão: as três técnicas de avaliação

[Aula 02] O **benchmarking** de hoje entra dentro da aferição, no ramo da carga de trabalho.



Revisão: qual técnica usar, e quando combinar

[Aula 02 — a decomposição da demanda é conteúdo novo, desta Aula 07]

Critério	Modelagem	Simulação	Aferição
Estágio do sistema	qualquer (mesmo inexistente)	qualquer	existente / protótipo
Custo	baixo	médio	alto
Precisão	moderada	moderada–alta	alta
Rapidez	alta	média	baixa
Complexidade de uso	alta (matemática)	média	alta

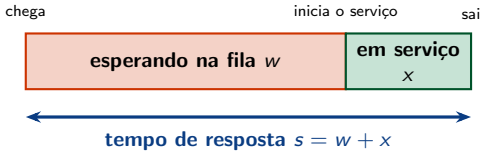
Modo híbrido: usar duas ou mais técnicas para verificação e validação cruzada. No servidor Web de aulas, a **modelagem** decompõe a demanda e a **medição** obtém cada parcela:

$$D_{servidor} = D_{CPU,recep} + D_{E/S} + D_{CPU,envio}$$

Fonte: Jain, cap. 3; pdfcoffee, cap. 11, p. 161-162.

Revisão: a fila por dentro

[Aula 03]



$\lambda = 1/E[t]$	taxa de chegadas (inverso do tempo entre chegadas)
$\mu = 1/E[x]$	taxa de serviço de **um** servidor; com m servidores, $m\mu$
n, n_w, n_s	no sistema, esperando na fila, em serviço



Figura reutilizada da Aula 05.

Revisão: Lei de Little e as relações

[Aulas 02 e 03]

No sistema inteiro

$$E[n] = \lambda \cdot E[s]$$

Só na fila

$$E[n_w] = \lambda \cdot E[w]$$

Vale para **qualquer sistema em equilíbrio**, desde que ninguém seja perdido por falta de *buffer*.

Exemplo da Aula 02

$\lambda = 50$ req/s e $E[s] = 0{,}2$ s

$$E[n] = 50 \times 0{,}2 = \mathbf{10} \text{ requisições}$$

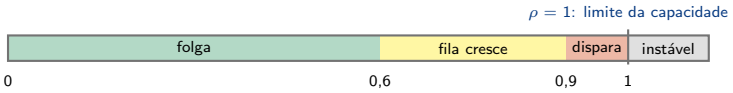
Se o sistema fica mais lento e $E[s] = 0{,}4$ s, então $E[n] = 20$ — o dobro acumulado, sem mudar a taxa de chegada.

As relações para guardar: $E[n] = E[n_w] + E[n_s]$ · $E[s] = E[w] + E[x]$ · $E[n] = \lambda E[s]$ · $\rho = \lambda/(m\mu) < 1$

Revisão: ρ, estabilidade e a cilada da utilização

[Aula 03]

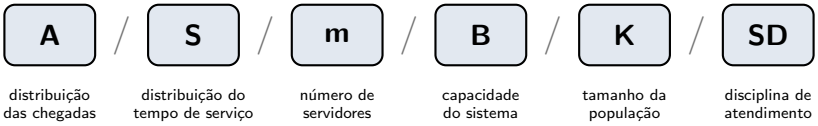
$$\rho = \frac{\text{taxa de chegadas}}{\text{taxa de serviço}} = \frac{\lambda}{\mu} \qquad \text{e, com } m \text{ servidores,} \quad \rho = \frac{\lambda}{m\mu}$$



- **Condição de estabilidade:** $\lambda < m\mu$, ou seja, $\rho < 1$ — só assim existe regime estacionário. (Com fila finita não há instabilidade: quem não cabe é recusado.)
- Entre 0,9 e 1, um pequeno aumento de λ **multiplica** a espera.

Revisão: notação de Kendall

[Aula 03]



Regra prática que cai em prova

Os três últimos são **opcionais**. Omitido **B**, a capacidade é ilimitada; omitido **K**, a população é infinita; omitido **SD**, a disciplina é FIFO.

Logo, **M/M/1** = chegadas de Poisson, serviço exponencial, 1 servidor, fila infinita, população infinita e FIFO.

SD, as disciplinas da Aula 03: FIFO/FCFS (ordem de chegada), **LIFO/LCFS** (reversa), **SIRO** (randômica), **RR** (fatia de tempo), **PS** (fatias infinitesimais) e **IS** (nunca se forma fila).

Revisão: as fórmulas da M/M/1

[Aula 03]

Do equilíbrio entre cortes, $\lambda p_n = \mu p_{n+1}$, sai $p_n = \rho^n p_0$; normalizando, $p_0 = 1 - \rho$:

$$\boxed{p_n = (1-\rho)\,\rho^n}$$

Grandeza	**Fórmula**	**Leitura**
Sistema vazio	$p_0 = 1 - \rho$	fração do tempo ocioso
Utilização	$U = 1 - p_0 = \rho$	fração do tempo ocupado
Nº médio no sistema	$E[n] = \rho/(1-\rho)$	inclui quem está em serviço
Nº médio na fila	$E[n_w] = E[n] - U$	só quem espera
Tempo de resposta	$E[s] = 1/[\mu(1-\rho)]$	espera + serviço
Tempo de espera	$E[w] = E[s] - 1/\mu$	tira o atendimento
k ou mais no sistema	$P(\geq k) = \rho^k$	$P(\geq 1) = U$

Ideia-chave: Tudo sai de um número só: o ρ.

Revisão: exercício resolvido de ponta a ponta

[Aula 03] Enunciado: um servidor Web recebe **3600 requisições por minuto** e atende **80 requisições por segundo**. Modele como M/M/1 e obtenha as métricas.

Passo 1 — mesma unidade: $\lambda = 3600/60 = \mathbf{60}$ req/s e $\mu = \mathbf{80}$ req/s. **Passo 2:** $\rho = 60/80 = \mathbf{0{,}75} < 1$, fila estável e $U = \mathbf{75\%}$.

Métrica	Conta	Resposta
Sistema vazio p_0	$1 - \rho = 1 - 0{,}75$	**25%**
Nº médio no sistema $E[n]$	$\rho/(1-\rho) = 0{,}75/0{,}25$	**3 req.**
Nº médio na fila $E[n_w]$	$E[n] - U = 3 - 0{,}75$	**2,25 req.**
Tempo de resposta $E[s]$	$1/[80(1-0{,}75)] = 1/20$	**0,05 s**
Tempo de espera $E[w]$	$E[s] - 1/\mu = 0{,}05 - 0{,}0125$	**0,0375 s**

Confira por Little: $E[n] = \lambda E[s] = 60 \times 0{,}05 = 3$ — bate com $\rho/(1-\rho)$.

Revisão: a mecânica da simulação

[Aulas 04 e 05]

Como o tempo passa: dois jeitos

Passos fixos (*time-driven*): avança de Δt em Δt e gasta CPU em passos vazios. **Salto de evento em evento**: pula direto para o próximo evento. É o jeito que usamos.

O laço de eventos

Relógio + **FEL** (*future event list*) + **estado**. Eventos **agendados** entram na FEL (chegada, fim de serviço); **condicionais** ocorrem quando a condição se cumpre. A rodada acaba quando a FEL esvazia.



Aula 04: errar no simulador custa quase nada.

Regra 40/20/40 (Shannon, 1989): **40%** modelar, **20%** programar, **40%** verificar e validar — programar é a menor parte.

Revisão: dá para confiar no simulador?

[Aulas 04 e 05]

Verificação

Construímos **certo***?* O programa implementa o modelo corretamente. É depuração.

Validação

Construímos o **certo***?* O modelo representa o sistema real. Compara-se com medição ou com o analítico.

A semente e as réplicas

O computador **não sorteia, calcula**: o **PRNG** segue uma receita determinística a partir da semente x_0, e `expovariate` vira tempo exponencial. Mesma semente, mesma sequência, mesmo resultado. Só trocando a semente, a espera média foi de **3,37 min** para **5,91 min**: 75% a mais.

Regra: nunca informe um resultado sem dizer quantas rodadas e quais sementes.

Prefira simuladores já validados pela comunidade (SimPy, ns-3, OMNeT++, Arena) e combine técnicas.

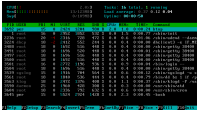
Revisão: aferição e monitores

[Aula 06]

Aferir tem dois ingredientes: a **carga** e o **monitor** que a registra.

Conceito	**O que é**
Evento	Qualquer mudança de estado do sistema
Traçado (*trace*)	O registro da sequência de eventos
Overhead	O que o monitor consome e perturba
Resolução	O menor detalhe visível ao monitor

Classificar um monitor, em três perguntas: (1) hardware, software, firmware ou híbrido? (2) por **evento** (exato) ou **amostragem** (estimativa)? (3) **on-line** ou **batch**?
Quatro critérios da carga: serviços exercitados, nível de detalhe, representatividade, atualidade.



Monitor de **software**: flexível e barato, paga em *overhead*.



Monitor de **hardware**: mede de fora, não pesa no sistema.

Guia de estudo para a avaliação

Tema	Saber explicar	Saber fazer
Conceitos (A01-02)	Métrica, parâmetro, carga; os passos do estudo	Classificar o que é métrica e o que é parâmetro num cenário
Técnicas (A02)	As três técnicas, vantagens, limites e o híbrido	Escolher a técnica dado o estágio do sistema e o custo
Filas (A03)	λ, μ, w, x, s, n; Kendall; estabilidade	Calcular ρ, p_n, U, $E[n]$, $E[n_w]$, $E[s]$, $E[w]$ e conferir por Little
Simulação (A04-05)	Evento, FEL, relógio; verificação $\times$ validação	Percorrer o laço de eventos à mão e ler a saída do script
Aferição (A06)	Carga real $\times$ sintética; tipos de monitor; *overhead*	Classificar um monitor pelas três perguntas
Benchmarking (A07)	O que LINPACK e STREAM calculam; SPEC; *ratio game*	Calcular MFLOPS; escolher a ferramenta pelo recurso

Leitura: Johnson & Coutinho, caps. 1, 2, 8 e 9; Jain, caps. 2, 3, 9, 16 e 25.

Ideia-chave: Calcular a M/M/1 e justificar a técnica escolhida: é isso que a prova cobra.

Próxima aula: a avaliação

29 de setembro de 2026 — Avaliação da Unidade I

A próxima aula é a prova, sobre tudo o que revisamos hoje.

- Leve calculadora. As contas da M/M/1 são simples, mas são contas.
- Refaça o exercício resolvido do servidor Web (M/M/1) com outros números.
- Revise a sua atividade de simulação: ela cobra os conceitos das Aulas 04 e 05.

Depois da avaliação, em **06 de outubro de 2026**, começa a **Unidade II** com **cadeias de Markov a tempo discreto**: a base formal por trás da teoria de filas que usamos de modo intuitivo na Aula 03.

Referências


JAIN, R. *The Art of Computer Systems Performance Analysis*. John Wiley & Sons, 1991.


JOHNSON, T. M. S.; COUTINHO, F. *Avaliação de desempenho de sistemas computacionais*. TrioStudio, 2011.


BOLCH, G.; GREINER, S.; DE MEER, H.; TRIVEDI, K. S. *Queuing Networks and Markov Chains*. Wiley & Sons, 2006.


MENASCÉ, D. A.; DOWDY, L. W.; ALMEIDA, V. A. F. *Performance by Design*. Prentice Hall, 2004.


KANT, K. *Introduction to Computer Systems Performance Evaluation*. McGraw-Hill, 1992.


STANDARD PERFORMANCE EVALUATION CORPORATION. `http://www.spec.org`.